

SPARSIFICATION OF 3D GAUSSIAN SPLATTING:
PRUNING INTERACTIONS, RECOVERY, AND POPULATION REFRESH

by

Matthew Mammano

A THESIS

Submitted to the Faculty of the Stevens Institute of Technology
in partial fulfillment of the requirements for the degree of

MASTER OF SCIENCE – MACHINE LEARNING

Matthew Mammano, Candidate

ADVISORY COMMITTEE

Philippos Mordohai, Advisor Date

Enrique Dunn, Reader Date

STEVENS INSTITUTE OF TECHNOLOGY
Castle Point on Hudson
Hoboken, NJ 07030
2026

SPARSIFICATION OF 3D GAUSSIAN SPLATTING:
PRUNING INTERACTIONS, RECOVERY, AND POPULATION REFRESH

ABSTRACT

3D Gaussian Splatting enables real-time novel-view synthesis but can produce millions of primitives per scene. Sparsification methods commonly score Gaussians independently, remove the lowest-ranked subset, and optimize the survivors. This thesis analyzes how importance scoring, iterative pruning, recovery optimization, and population refresh affect quality under an exact Gaussian budget. Using GaussianPOP as the pruning baseline, we evaluate a ground-truth-aware signed removal score, pruning and recovery schedules, and population refresh. Population refresh uses standard adaptive density control to temporarily triple the Gaussian count before pruning back to the original budget. Paired experiments on five Tanks and Temples scenes match the source checkpoint, data split, optimizer-update count, and final Gaussian count. After four refresh cycles and a common 100,000-update continuation, refresh improves the five-scene mean by .3028 dB PSNR and .00382 SSIM and reduces LPIPS by .00286 relative to fixed-population optimization, improving 14 of 15 scene-metric comparisons. These results show that population refresh can reach a better fixed-count solution.

Author: Matthew Mammano

Advisor: Philippos Mordohai

Date: August 26, 2026

Department: Computer Science

Degree: Master of Science – Machine Learning

Acknowledgments

I am thankful to my advisor, Professor Philippos Mordohai, for his guidance and critical feedback throughout this work. I am also thankful to Professor Enrique Dunn for the time and expertise he contributed as my reader. I was fortunate to take courses with both professors at Stevens. I thank the Stevens Institute of Technology for providing the computing resources used in these experiments. Finally, I thank my loving girlfriend, my family, and my friends for their constant encouragement and support.

Table of Contents

Abstract	iii
Acknowledgments	iv
List of Tables	viii
List of Figures	ix
1 Introduction	1
2 Background	4
2.1 Novel View Synthesis Problem Setup	4
2.2 Neural Radiance Fields	5
2.3 3D Gaussian Splatting	7
2.3.1 Gaussian Primitives	7
2.3.2 Rendering: Projection and Rasterization	8
2.3.3 Optimization and Adaptive Density Control	10
2.4 GaussianPOP	12
2.4.1 Error-Quantification Criterion	13
2.4.2 Pruning Schedule	14
2.5 Evaluation Protocol	14
2.5.1 Per-Image Metrics	15
2.5.2 Gaussian Count and the Fixed-Budget Protocol	15
3 Related Work	16

3.1	Pruning by Importance	16
3.1.1	Contribution and Visibility Scores	17
3.1.2	Sensitivity Scores	18
3.1.3	Sparsity-Induction Training	18
3.2	Densification and Placement	19
3.3	Storage Compression	21
4	Method	22
4.1	Backbone and Initialization	22
4.2	Population Refresh	23
4.3	Signed Removal Score	25
4.4	Pruning Schedule	26
4.5	Parent Selection During Refresh	28
5	Results	29
5.1	Population Refresh Against a Fixed Population	29
5.2	Spatial Occupancy Analysis	32
5.3	Peak Population and Cycle Count	34
5.3.1	Peak Size	34
5.3.2	Cycle Count	34
5.4	Pruning Components	35
5.4.1	Signed Removal Score	35
5.4.2	Rescore Frequency	36
5.4.3	Staged Pruning and Recovery	37
5.4.4	Visibility Priority and View Aggregation	38
5.5	Parent Selection During Refresh	39

5.6	Summary of Findings	40
6	Discussion and Conclusion	42
6.1	Discussion	42
6.2	Future Work	42
6.3	Conclusion	43
	Bibliography	44
	Vita	47

List of Tables

3.1	Pruning criteria by score family	17
3.2	Densification signals and schedules	20
4.1	Per-scene initialization and Gaussian budget	23
5.1	Fixed baseline against population refresh after refinement	30
5.2	Per-cell quality change grouped by occupancy change	33
5.3	Effect of refresh peak size	34
5.4	Signed removal score against GaussianPOP across the residual weight	36
5.5	Rescore frequency before and after recovery	37
5.6	Four-stage against one-stage pruning at equal recovery budget	38
5.7	Minimum visibility threshold for deletion priority	39
5.8	Deterministic and probabilistic parent selection	40
5.9	Summary of controlled comparisons	41

List of Figures

1.1	Matched-count qualitative comparison	2
2.1	3D Gaussian Splatting rendering pipeline	8
4.1	Population-refresh schedule	24
4.2	End-only and staged pruning schedules	27
5.1	Matched population-refresh trajectory	31
5.2	Qualitative population-refresh comparisons	32
5.3	Projected occupancy change in one held-out view	33
5.4	Complete fifteen-cycle refresh trajectory	35

Chapter 1

Introduction

Creating a navigable representation of a real scene from ordinary images is a long-standing problem in computer vision and graphics. Novel view synthesis (NVS) reconstructs a scene from photographs and renders viewpoints absent from the capture. Applications include city-scale reconstruction and autonomous-driving simulation [1] and photorealistic avatars for telepresence and augmented or virtual reality [2].

Neural Radiance Fields (NeRFs) [3] represent a scene as a continuous function of position and viewing direction. A multilayer perceptron predicts color and density at samples along each camera ray, enabling differentiable volume rendering but making rendering costly.

3D Gaussian Splatting (3DGS) [4] instead uses explicit anisotropic Gaussians with learned position, shape, opacity, and view-dependent appearance. Projecting and alpha-compositing these primitives supports real-time rendering, but adaptive density control can produce one to five million Gaussians without enforcing a final population budget.

Large, scene-dependent populations increase storage and rendering work [5, 6]. This motivates *sparsification*: reducing Gaussian count while preserving reconstruction quality at an exact population budget.

GaussianPOP [7] analytically scores the rendered-image change caused by removing each Gaussian. Its long-tailed scores indicate many low-impact primitives, but bulk removal changes transmittance and the contributions of survivors. The result may therefore depend on rescore frequency, recovery, and whether growth can replace poorly positioned primitives.

This thesis evaluates these choices through paired comparisons on five Tanks and Temples scenes [8], matching checkpoints, optimizer updates, and final Gaussian counts. Rescore frequency affects immediate pruning damage but little after recovery. Population refresh grows $N \rightarrow 3N$ and prunes back to N . After four cycles and a common 100,000-update continuation, refresh improves the five-scene mean by .3028 dB PSNR and .00382 SSIM, reduces LPIPS by .00286, and improves 14 of 15 scene-metric comparisons relative to fixed-population optimization. Figure 1.1 shows the Meetingroom comparison.



Figure 1.1: Meetingroom held-out view 27 at matched final Gaussian count and optimizer-update count. Boxes mark the cropped region.

None of the tested scoring, visibility, or parent-selection variants improves PSNR, SSIM, and LPIPS together.

Our contributions are:

1. A ground-truth-aware signed extension of GaussianPOP’s removal score.
2. Matched analyses of rescore frequency and recovery placement.
3. Exact-count evaluation of refresh peak size, parent selection, and cycle count.

Chapter 2 introduces NVS, NeRF, 3DGS, GaussianPOP, and the evaluation protocol. Chapter 3 reviews Gaussian pruning, densification and placement, and storage compression. Chapter 4 presents the signed removal score and the pruning, recovery, and population-refresh protocols. Chapter 5 reports the controlled experiments. Chapter 6 discusses limitations, future work, and conclusions.

Chapter 2

Background

This chapter introduces the novel view synthesis problem and notation, motivates the shift from implicit neural radiance fields to explicit Gaussian primitives, and describes 3DGS and GaussianPOP pruning. It closes with the evaluation metrics and fixed-count protocol used throughout this thesis.

2.1 Novel View Synthesis Problem Setup

Novel view synthesis constructs renderings of unobserved viewpoints given a set of images. This thesis considers static scenes only; dynamic and deformable scenes are out of scope. Each camera i has a pose that decomposes into a rotation $\mathbf{R}_i$ and a translation $\mathbf{t}_i$,

$$(\mathbf{R}_i, \mathbf{t}_i) \in SE(3), \quad (2.1)$$

where $SE(3)$ is the Special Euclidean group of rigid motions, whose rotation part

$$\mathbf{R}_i \in SO(3), \quad \mathbf{t}_i \in \mathbb{R}^3, \quad (2.2)$$

belongs to the Special Orthogonal group $SO(3)$, with $\mathbf{R}_i^\top \mathbf{R}_i = \mathbf{I}_3$ and $\det(\mathbf{R}_i) = 1$.

A camera intrinsic matrix

$$\mathbf{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \quad (2.3)$$

maps 3D points in camera coordinates into 2D pixels, with f_x, f_y focal lengths and (c_x, c_y) principal point offset, typically defined as the image center.

A rendering function $\hat{C}(\mathbf{p}; \Theta)$ maps a pixel coordinate $\mathbf{p}$ and a set of scene parameters Θ to a color. The role of $\hat{C}$ is shared across methods: it turns a viewing direction and scene parameters into a rendered color, though the parameterization of Θ varies widely, from the weights of a neural network to a set of geometric primitive attributes.

The standard pipeline begins with a multi-view image set whose poses are recovered by Structure-from-Motion (SfM), typically COLMAP [9], and split into training and test views. Methods diverge in the choice of scene representation, either implicit, such as a network, or explicit, such as a primitive set. In either case a differentiable renderer produces each image, so gradients flow back to Θ and the parameters are optimized against a photometric loss. Writing $\hat{I}_i(\Theta)$ for the image that camera i renders under Θ , formed by evaluating $\hat{C}(\cdot; \Theta)$ at each pixel, the objective is

$$\Theta^* = \arg \min_{\Theta} \sum_i \mathcal{L}(\hat{I}_i(\Theta), I_i). \quad (2.4)$$

Here $\mathcal{L}$ is the photometric loss, and I_i is the captured image at camera i .

2.2 Neural Radiance Fields

Neural Radiance Fields (NeRF) represent scenes using a continuous 5D function approximated by a multilayer perceptron (MLP) [3]. The network $F_{\Theta} : (\mathbf{x}, \mathbf{d}) \rightarrow (\mathbf{c}, \sigma)$ takes a 3D location $\mathbf{x} = (x, y, z)$ and a 2D viewing direction $\mathbf{d} = (\theta, \phi)$, and returns a view-dependent color $\mathbf{c} = (r, g, b)$ and a volumetric density σ . Density is constant across viewing directions, enabling consistent geometry across views while allowing direction-based color to handle view-dependent effects such as specular highlights.

To form an image, a ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ is cast through each pixel and radiance is accumulated along it. Following classical volume rendering [10], the expected color between near and far bounds t_n and t_f is

$$\begin{aligned} C(\mathbf{r}) &= \int_{t_n}^{t_f} T(t) \sigma(\mathbf{r}(t)) \mathbf{c}(\mathbf{r}(t), \mathbf{d}) dt, \\ T(t) &= \exp\left(-\int_{t_n}^t \sigma(\mathbf{r}(s)) ds\right). \end{aligned} \tag{2.5}$$

Here σ is density, $\mathbf{c}$ is view-dependent color, and $T(t)$ is accumulated transmittance: the probability that the ray travels from t_n to t without being blocked. While the integral has no analytic form, NeRF approximates it using numerical quadrature over samples drawn stratified along the ray, giving the discrete estimate

$$\begin{aligned} \hat{C}(\mathbf{r}) &= \sum_k T_k \alpha_k \mathbf{c}_k, \\ \alpha_k &= 1 - \exp(-\sigma_k \delta_k), \\ T_k &= \prod_{j < k} (1 - \alpha_j). \end{aligned} \tag{2.6}$$

The sample spacing is $\delta_k = t_{k+1} - t_k$, α_k is the opacity induced by density σ_k over that interval, and $T_k = \exp(-\sum_{j < k} \sigma_j \delta_j)$ is equivalent to the product form in Equation 2.6. This estimate and 3DGS use the same front-to-back alpha-compositing form. NeRF obtains α_k from density integrated over a ray interval, whereas 3DGS multiplies stored opacity by a projected 2D footprint, as Section 2.3.2 describes.

While NeRF enables high-quality NVS, its computational cost is the main disadvantage. Producing one pixel takes hundreds of MLP evaluations at samples along its ray, and an image has millions of pixels. This per-sample network cost rules out real-time rendering and motivates the more explicit representation of the next

section.

2.3 3D Gaussian Splatting

3D Gaussian Splatting (3DGS) replaces the implicit, per-ray-sampled NeRF field with an explicit set of anisotropic 3D primitives that are rasterized rather than integrated along rays [4]. 3DGS matches or modestly exceeds the synthesis quality of prior state-of-the-art methods on standard benchmarks according to common metrics like PSNR and SSIM, while taking minutes instead of hours to train and rendering in real time.

2.3.1 Gaussian Primitives

Each primitive is a splat: a 3D spatial footprint together with the appearance it contributes to a view. Its footprint is a 3D Gaussian,

$$G(\mathbf{x}) = \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})\right), \quad (2.7)$$

positioned by a mean $\boldsymbol{\mu} \in \mathbb{R}^3$ and shaped by a covariance $\boldsymbol{\Sigma} \in \mathbb{R}^{3 \times 3}$. Rather than optimize $\boldsymbol{\Sigma}$ directly, 3DGS factors the covariance as

$$\boldsymbol{\Sigma} = \mathbf{R} \mathbf{S} \mathbf{S}^\top \mathbf{R}^\top, \quad (2.8)$$

into a rotation $\mathbf{R}$, stored as a quaternion, and a diagonal scale $\mathbf{S}$. Section 2.3.3 explains why this factored form, rather than the six raw entries of $\boldsymbol{\Sigma}$, is what training updates.

Beyond shape and position, primitives carry an opacity $\alpha \in [0, 1]$ and color

using spherical harmonic (SH) coefficients, yielding view-dependent appearances:

$$\mathbf{c}(\mathbf{d}) = \sum_{l=0}^L \sum_{m=-l}^l \mathbf{k}_l^m Y_l^m(\mathbf{d}). \quad (2.9)$$

The Y_l^m are fixed SH basis functions, the $\mathbf{k}_l^m$ are learned per-Gaussian coefficients, $\mathbf{d}$ is the viewing direction, and the band limit L is capped at 3 in 3DGS. The DC term, where $l = 0$, acts as a base color, and higher bands add low-frequency view-dependent shading on top. Compared with view-independent color, the SH representation captures smooth view-dependent shading but not sharp specular effects.

A band limit of $L = 3$ spans four SH bands, or $(L + 1)^2 = 16$ coefficients per color channel and 48 across the three channels. Counting every attribute, one primitive stores a mean with 3 values, a rotation quaternion with 4, a scale with 3, an opacity with 1, and these 48 SH coefficients: 59 floating-point values in all. The SH block alone is 81 percent of those values, so per-primitive storage is dominated by view-dependent color.

2.3.2 Rendering: Projection and Rasterization

Rendering in 3DGS projects each 3D Gaussian into screen space, sorts the projected splats by depth, and alpha-composites their colors per pixel [4].

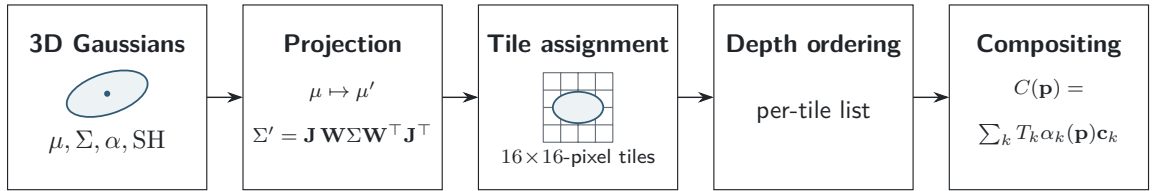


Figure 2.1: Each Gaussian is projected into screen space, assigned to the tiles its footprint overlaps, depth sorted within the tile lists, and alpha composited per pixel.

Projection. Before a primitive can be rasterized, its position and shape must be projected from 3D world coordinates into the image. The mean transforms into camera coordinates through the full camera model, $\mu' = \mathbf{W}\mu$ followed by $\mathbf{K}$ and the perspective division. The covariance is less simple: because perspective projection is nonlinear, the projected footprint of a 3D Gaussian is not itself a 2D Gaussian. Since Gaussians are preserved only under affine maps, 3DGS linearizes the projection at the mean with its Jacobian $\mathbf{J}$, following the EWA splatting approximation [11], giving

$$\Sigma' = \mathbf{J} \mathbf{W} \Sigma \mathbf{W}^\top \mathbf{J}^\top. \quad (2.10)$$

The 2D screen-space covariance is read off by dropping the last row and column of Σ' .

Sorting and tiling. Screen space is divided into 16×16 tiles. Each Gaussian is tagged with the tiles its footprint overlaps, discarding any Gaussian whose 99% confidence footprint falls outside the view frustum. The tagged instances are sorted globally by tile and view-space depth, producing a depth-ordered list for each tile. This ordering approximates the exact per-pixel depth order, trading exactness for speed.

Rasterization and alpha blending. Per pixel, the sorted list is traversed front-to-back. At a pixel $\mathbf{p}$, each primitive contributes a per-pixel alpha equal to its learned opacity times its projected 2D Gaussian evaluated there,

$$\begin{aligned} \alpha_k(\mathbf{p}) &= \alpha_k G'_k(\mathbf{p}), \\ G'_k(\mathbf{p}) &= \exp\left(-\frac{1}{2}(\mathbf{p} - \mu'_k)^\top (\Sigma'_k)^{-1}(\mathbf{p} - \mu'_k)\right). \end{aligned} \quad (2.11)$$

The scalar $\alpha_k \in [0, 1]$ is the learned opacity of primitive k , G'_k is its projected 2D Gaussian, and μ'_k, Σ'_k are its projected 2D mean and covariance from Equation 2.10. Colors come from the SH coefficients in Equation 2.9, and the pixel color accumulates by front-to-back α -compositing,

$$\begin{aligned} C(\mathbf{p}) &= \sum_k \mathbf{c}_k \alpha_k(\mathbf{p}) T_k(\mathbf{p}), \\ T_k(\mathbf{p}) &= \prod_{j < k} (1 - \alpha_j(\mathbf{p})). \end{aligned} \tag{2.12}$$

The transmittance $T_k(\mathbf{p})$ is the fraction of light still unblocked between primitive k and the camera: a partly transparent primitive lets background light through, while an opaque one blocks it. Blending in 3DGS terminates early once accumulated opacity saturates, and the whole pass runs per pixel in parallel on the GPU, delivering the real-time rates reported by Kerbl et al. [4].

2.3.3 Optimization and Adaptive Density Control

One isotropic Gaussian is initialized at each point of the sparse SfM point cloud of Section 2.1. Training then refines its position, scale, rotation, color, and opacity across viewpoints.

Each operation in 3DGS, including projection, Gaussian evaluation, and alpha blending, is differentiable, so the gradient of the loss with respect to each parameter exists. Adam [12] updates each of these parameters to minimize the objective of Equation 2.4.

The loss combines an $\mathcal{L}_1$ term with a structural-dissimilarity (D-SSIM) term,

$$\mathcal{L} = (1 - \lambda) \mathcal{L}_1 + \lambda \mathcal{L}_{\text{D-SSIM}}. \tag{2.13}$$

Here $\mathcal{L}_1$ is the mean absolute error between the rendered and ground-truth pixels, $\mathcal{L}_{\text{D-SSIM}} = (1 - \text{SSIM})/2$, SSIM is the Structural Similarity Index between the same image pair, and $\lambda = 0.2$ in 3DGS. Three quantities are reparameterized so that gradient descent stays in valid ranges: scale through an exponential, opacity through a sigmoid, and covariance through the $\mathbf{R}\mathbf{S}\mathbf{S}^\top\mathbf{R}^\top$ factorization of Equation 2.8. The covariance case matters most: a valid covariance must be positive semi-definite, meaning symmetric with no negative eigenvalues, and optimizing the six free entries of Σ directly would let updates drift into invalid matrices that no longer describe a real ellipsoid. For any $\mathbf{R}$ and $\mathbf{S}$, the factored product is provably positive semi-definite, so the constraint is baked into the parameterization rather than enforced during training.

Optimization alone cannot decide how many primitives a scene needs or where they belong, so 3DGS interleaves Adaptive Density Control (ADC), which edits the primitive set on a schedule during training:

- **Densify.** For primitive k , ADC averages the magnitude of the loss gradient with respect to its projected mean μ'_k over the recent iterations V_k in which it was visible. It selects the primitive when

$$\frac{1}{|V_k|} \sum_{i \in V_k} \|\nabla_{\mu'_k} \mathcal{L}_i\|_2 > \tau_{\text{pos}}, \quad (2.14)$$

where τ_{pos} is the densification threshold. ADC clones a selected small Gaussian or replaces a selected large Gaussian with smaller samples from its spatial distribution.

- **Prune.** Remove primitives with opacity below ϵ_α or excessive screen-space or world-space size.
- **Opacity reset.** Periodically reduce opacity globally. Useful primitives recover

through optimization, while persistently weak primitives are later pruned.

ADC alternates parameter optimization with densification, pruning, and opacity reset. Chapter 3 reviews methods that modify how densification selects and places new primitives.

2.4 GaussianPOP

After training, a 3DGS scene can contain millions of primitives. Rendering work generally increases with population because each visible Gaussian is projected, assigned to the tiles it overlaps, sorted, and alpha-composited at the pixels it covers [4]. Reducing the population can therefore lower storage and rendering work. Post-hoc pruning methods generally rank primitives by visible contribution or removal sensitivity, while training-time methods can instead learn masks or impose sparsity constraints.

Contribution methods rank primitives using opacity or blending weight, sometimes combined with volume [4, 13, 5, 14]. These scores measure rendered visibility without reference images. Sensitivity methods instead estimate how the render or reconstruction error changes when a primitive is perturbed or removed [15, 16]. Chapter 3 compares these criteria.

GaussianPOP is a contribution method whose criterion is computed from the render alone, without ground-truth input [7]. What sets it apart from LightGaussian and RadSplat is that it measures the exact change to the render that removing a primitive would cause, rather than a blending-weight proxy for that change. The next subsection derives this quantity; the derivation produces a color-contrast factor ($\mathbf{c}_k - \mathbf{b}_{k+1}$) that blending-weight scores lack, so a primitive that is opaque and frontmost but whose color already matches its background scores zero under GaussianPOP even though a blending-weight criterion would rank it highly.

2.4.1 Error-Quantification Criterion

Following GaussianPOP, consider the alpha-blended color at a pixel, isolating a single Gaussian k :

$$C_{\text{render}} = P_{k-1} + T_k \alpha_k \mathbf{c}_k + T_k (1 - \alpha_k) \mathbf{b}_{k+1}, \quad (2.15)$$

where P_{k-1} is the color already accumulated in front of k , $T_k = \prod_{j < k} (1 - \alpha_j)$ is the transmittance reaching k , α_k is the opacity contribution of k , $\mathbf{c}_k$ is the color of k , and $\mathbf{b}_{k+1}$ is the background color accumulated behind k . Removing only Gaussian k from that pixel leaves

$$C'_{\text{render}} = P_{k-1} + T_k \mathbf{b}_{k+1}. \quad (2.16)$$

The squared change to the rendered pixel caused by removing k is therefore the difference of the two expressions above,

$$\Delta \text{SE}_k = \|C_{\text{render}} - C'_{\text{render}}\|^2 = \|T_k \alpha_k (\mathbf{c}_k - \mathbf{b}_{k+1})\|^2. \quad (2.17)$$

The derivation yields the color-contrast factor $(\mathbf{c}_k - \mathbf{b}_{k+1})$ whose advantage was described in Section 2.4. Because ΔSE_k is read from the render, not the reference photograph, this thesis treats it as a removal-impact, or contribution, score.

Computing this per primitive naively would need a separate pass for every Gaussian and view. GaussianPOP computes the individual removal score for every Gaussian in a single forward render. For Gaussian k , the score measures how the rendered image would change if only k were removed and every other Gaussian remained unchanged. The per-pixel values are summed over all training views with

equal weight, so the importance of each primitive is

$$S_k = \sum_{\text{views}} \sum_{\text{pixels}} \Delta \text{SE}_k, \quad (2.18)$$

where S_k is the aggregate importance score for primitive k . Pruning by this score is followed by a fine-tuning stage. This thesis evaluates how these individual scores behave when Gaussians are removed in batches, scores are recomputed between batches, and the survivors are optimized.

2.4.2 Pruning Schedule

GaussianPOP applies this criterion during or after training [7]. In the 30,000-iteration on-training schedule, configurable fractions are removed at iterations 15,000 and 20,000, with fine-tuning after each event; the paper also reports a 40,000-iteration variant. The post-training schedule instead starts from the completed model, performs eight error re-quantification cycles, and optionally adds 5,000 fine-tuning iterations.

2.5 Evaluation Protocol

Tanks and Temples [8] is a benchmark of indoor and outdoor scenes captured under realistic conditions for image-based 3D reconstruction. This thesis evaluates five scenes from the benchmark: Barn, Caterpillar, Ignatius, Meetingroom, and Truck.

Unless otherwise stated, every quantitative result uses held-out test views. Following the original 3DGS evaluation protocol, we sort images by filename, assign images whose zero-based index satisfies $\text{index} \bmod 8 = 0$ to the test set, and use the remaining images for training [4]. This split is fixed across all compared methods.

2.5.1 Per-Image Metrics

Peak Signal-to-Noise Ratio (PSNR) measures pixelwise reconstruction error:

$$\text{PSNR} = 10 \log_{10} \left(\frac{\text{MAX}_I^2}{\text{MSE}} \right), \quad (2.19)$$

where MSE is the mean squared error between the rendered and reference images and MAX_I is the maximum pixel value. Higher PSNR is better.

Structural Similarity (SSIM) [17] compares local luminance, contrast, and structure:

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + c_1)(2\sigma_{xy} + c_2)}{(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)}, \quad (2.20)$$

where x and y are the rendered and reference images, μ and σ^2 denote local means and variances, σ_{xy} is their local covariance, and c_1, c_2 stabilize the division. Higher SSIM is better.

Learned Perceptual Image Patch Similarity (LPIPS) [18] measures perceptual distance using deep-network features. Lower LPIPS is better. We report all three metrics because no single metric fully characterizes reconstruction quality.

2.5.2 Gaussian Count and the Fixed-Budget Protocol

This thesis compares methods by Gaussian count, not only by bytes on disk. The reason is that fewer primitives leave more headroom for the downstream compression codecs that operate per Gaussian, such as grid layouts [19] and vector quantization [20]. Accordingly, all comparisons use a matched-count protocol: quality metrics are compared only at an exactly matched number of Gaussians across methods, which removes count as a confounding variable.

Chapter 3

Related Work

3DGS compression can reduce either the number of Gaussians or the storage required by each Gaussian [7, 15]. This chapter first reviews methods that reduce Gaussian count through pruning or sparsity-inducing training. It then reviews densification and placement because population refresh temporarily adds Gaussians before pruning the model back to its exact target count. The final section covers methods that encode each remaining Gaussian more compactly. This thesis focuses on pruning and fixed-count population refresh, a post-training grow-prune update that temporarily increases the population and then returns it to its initial count.

3.1 Pruning by Importance

The largest body of work removes primitives from a trained model by ranking them with a per-primitive importance score and deleting the lowest-scoring fraction. The scores differ in what they measure and in whether they require the ground-truth image; Table 3.1 groups them.

Table 3.1: Pruning criteria differ primarily in whether they score visible contribution, removal sensitivity, or sparsity during training.

Method	Family	What it scores	GT-free	Post-hoc
3DGS [4]	opacity	α	yes	yes
Mini-Splatting [13]	contribution	$\sum T\alpha$	yes	yes
RadSplat [14]	contribution	$\max T\alpha$	yes	yes
LightGaussian [5]	contribution	contribution $\times$ volume $\gamma(\Sigma)$	yes	yes
MesonGS [21]	contribution	contribution $\times$ normalized volume	yes	yes
EAGLES [22]	contribution	per-pixel contribution	yes	yes
C3DGS [15]	sensitivity	$ \partial E / \partial p $	yes	yes
PUP 3D-GS [16]	sensitivity	Fisher/Hessian score	no	yes
GaussianSpa [23]	sparsity training	$\ a\ _0 \leq \kappa$ constraint	no	no
MaskGaussian [24]	sparsity training	learned mask probability	no	no
GaussianPOP [7]	analytic contribution	$\ T\alpha(c - b)\ ^2$	yes	yes

Table 3.1 follows the score comparison in GaussianPOP [7], while the family, ground-truth, and post-hoc classification is the organization in this thesis. GT-free marks criteria computable from the render alone, without the ground-truth image; post-hoc marks those applicable to an already-trained model rather than requiring a specialized training run.

3.1.1 Contribution and Visibility Scores

These scores rank a primitive by how much it appears in the rendered image, without reference to the ground truth. Vanilla 3DGS [4] prunes by opacity α alone. Mini-Splatting [13] sums the blending weight of each primitive $\sum T\alpha$ over the pixels it covers. RadSplat [14] instead keeps the single strongest ray contribution $\max T\alpha$, so that a primitive dominant in even one view survives. LightGaussian [5] weights

accumulated contribution by primitive volume when pruning. In its Room experiment, pruning and recovery reduce the representation from 365 MB to 123 MB before spherical-harmonic compression or vector quantization. MesonGS [21] scores by contribution times normalized volume. EAGLES [22] pairs a per-pixel contribution score with a lightweight quantized encoding of the survivors. GaussianPOP [7] belongs to this family but replaces the blending-weight proxy with the exact rendered-image change caused by removing a primitive, as described in Section 2.4.

One bias runs through the family. PUP 3D-GS [16] argues that visibility-based scores can favor larger primitives because a larger splat projects onto more pixels and accumulates more visibility. That critique matters here because a compression method that preserves large but redundant primitives may reach the target count while leaving fine structure underrepresented.

3.1.2 Sensitivity Scores

A second family ranks a primitive by how sensitive the render is to it, rather than by how much it contributes. C3DGS [15] uses the first-order sensitivity $|\partial E / \partial p|$, aggregated over the training views and computed from the render alone. PUP 3D-GS [16] instead approximates the second-order sensitivity of the $\mathcal{L}_2$ reconstruction error using Fisher/Hessian information, so it references the ground-truth images. Both are local linearizations around the converged model and do not require a fresh training run.

3.1.3 Sparsity-Induction Training

A third route folds compaction into training itself. GaussianSpa [23] formulates simplification as constrained optimization with a maximum primitive count, while

MaskGaussian [24] learns a probabilistic on/off mask for each primitive. In both, the compaction is inseparable from a from-scratch training run, so they neither yield a standalone score that could be reused for pruning nor fit the post-hoc setting of this thesis.

GaussianPOP [7] reports a long-tailed distribution of removal scores: most Gaussians have little estimated effect on the rendered image, while a small fraction have much larger effects. Many Gaussians are therefore clear low-impact removal candidates, and differences among criteria matter more as pruning becomes more aggressive. Where contribution scores use heuristic proxies and sensitivity scores use local approximations, GaussianPOP analytically computes the squared change in rendered color caused by removing one Gaussian, and recomputes those scores over multiple post-training pruning cycles. This thesis evaluates score recomputation frequency, multi-view aggregation, and recovery placement under an exact final Gaussian count.

3.2 Densification and Placement

Pruning only removes primitives. Densification is the opposite operation, deciding where primitives are added or moved, and standard ADC makes that decision from positional-gradient magnitude during training, before any compressor runs. The methods below revise how that decision is made; Table 3.2 lists the signal each one uses.

Table 3.2: Reviewed densification methods modify the signal or operator schedule used to add Gaussians.

Method	Signal driving placement	Signal type
3DGS ADC [4]	positional gradient	magnitude
Rota Bulò et al. [25]	per-pixel error	magnitude
Zeng et al. [26]	high-frequency content	magnitude
Taming-3DGS [6]	composite per-Gaussian score	importance
FreGS [27]	frequency-domain residual	magnitude
Huang et al. [28]	positional gradient	magnitude
Huang et al. [28]	split-then-clone ordering	operator schedule
SteepGS [29]	loss curvature at saddle points	curvature
GDAGS [30]	per-primitive gradient coherence	direction
LPM [31]	back-projected error regions	magnitude
EDGS [32]	dense-correspondence initialization	none

Prior work modifies different parts of ADC rather than simply replacing its densification threshold. Rota Bulò et al. [25] replace positional-gradient selection with a contribution-weighted per-pixel error score and limit the number of added Gaussians. Zeng et al. [26] retain gradient-based selection but use a dynamic threshold and constrain Gaussian scale using local density. FreGS [27] adds a progressively annealed frequency-domain loss, whose gradients guide densification from coarse to fine image content. Huang et al. [28] separate the split and clone schedule, using splits first for spatial coverage and introducing clones later for local refinement.

A few methods use richer densification signals. SteepGS [29] identifies primitives near saddle points using loss curvature. GDAGS [30] measures whether a primitive’s per-pixel position gradients agree in direction or cancel out. EDGS [32] avoids iterative densification by initializing a dense population from image correspondences. LPM [31] back-projects image-space rendering error into localized 3D regions and densifies within them. Taming-3DGS [6] instead samples primitives using a composite score during constructive, budget-constrained training.

The parent-selection experiment in this thesis retains the positional-gradient criterion and standard cloning and splitting, changing only how eligible parents are selected. Unlike prior densification methods, it applies this comparison after training within repeated grow–prune cycles that return the model to its original exact budget.

3.3 Storage Compression

A third family is orthogonal to pruning: it reduces the bytes stored per primitive while leaving the count fixed. CompGS [20] vector-quantizes per-primitive attributes using learned codebooks. Optimized Minimal 3DGS [33] represents each primitive using compact features and a small position-indexed neural field, then applies sub-vector quantization. Self-Organizing Gaussian Grids [19] arrange attributes on a 2D grid so that neighboring cells contain similar values and standard image codecs can be applied. These methods complement count reduction by reducing per-primitive storage, but they do not directly reduce the number of Gaussians projected, tile-assigned, sorted, and blended during rendering.

Chapter 4

Method

This chapter describes the backbone and initialization (RaDe-GS and GaussianPOP), population refresh, the signed removal score, pruning schedules, and parent selection.

4.1 Backbone and Initialization

All experiments extend RaDe-GS [34], which retains the 3DGS [4] primitive, but optimizes several operations as described in Section 2.3. Its depth-and-normal regularization is disabled in every reported run.

The backbone is trained once on each scene to produce the compressed checkpoint used by every later experiment on that scene. Base training runs for 40,000 iterations. Adaptive density control is active from iteration 500 to 15,000 with a positional-gradient threshold of 2×10^{-4} and a densification interval of 100 iterations, and opacity is reset every 3,000 iterations. GaussianPOP [7] then removes 73% of the population at iteration 20,000 and again at 25,000, so 7.29% of the peak survives the two passes, and a final optimization phase begins at iteration 38,000. A post-training GaussianPOP pass trims the remainder to the exact budget N .

Table 4.1 lists the five Tanks and Temples [8] scenes. Four are initialized from sparse Structure-from-Motion point clouds, while Caterpillar is initialized from a dense COLMAP [9] reconstruction. This mix tests the method across different initial reconstruction densities. All scenes undergo the same two scheduled pruning passes, followed by a trim to the exact budget N where required.

Table 4.1: Each experimental checkpoint is roughly 93–96% below its adaptive-density-control peak, so every comparison begins from an already-compressed model.

Scene	Initial points	ADC peak	After both prunes	Budget N
Barn	202,747	1,534,904	111,895	100,000
Caterpillar	10,871,729	3,483,583	253,954	200,000
Ignatius	155,306	4,903,841	357,491	300,000
Meetingroom	123,353	1,800,347	131,246	131,246
Truck	136,040	4,365,445	318,242	183,000

Unless stated otherwise, paired comparisons use the same source checkpoint, data split, rendering resolution, update budget, and final Gaussian count. Images are rendered at native resolution, and primitives below opacity 0.005 are discarded.

4.2 Population Refresh

Population refresh is a post-training grow–prune operation that expands a trained model and returns it to its original exact count. One cycle spans 40,000 optimizer updates. The fixed-population schedule keeps the original N primitives and uses the same number of updates without growth or pruning. Figure 4.1 summarizes the fixed-population and refresh schedules. The refresh cycle proceeds in four stages:

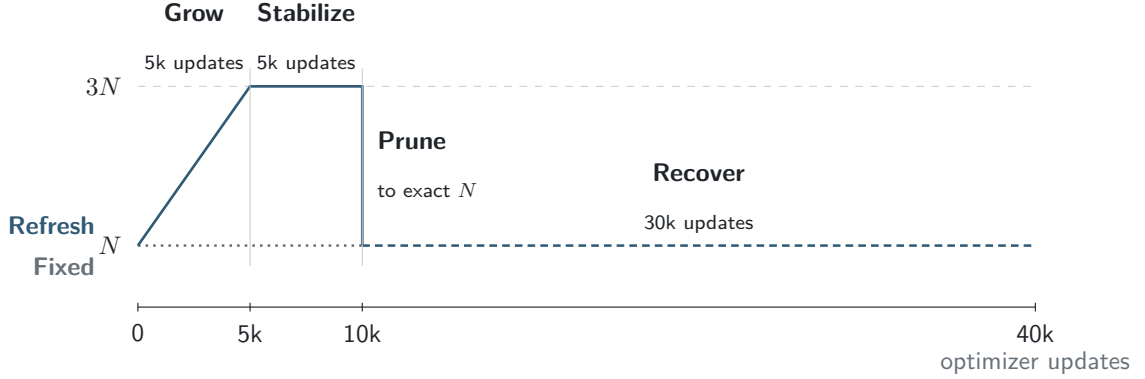


Figure 4.1: Refresh grows from N to $3N$, stabilizes, prunes to N , then shares the recovery path with the fixed-population schedule, which remains at N throughout.

1. **Grow.** Standard adaptive density control expands the population from N to $3N$ over 5,000 updates.
2. **Stabilize.** All primitives are optimized for another 5,000 updates before removal scores are computed.
3. **Prune.** GaussianPOP scores every primitive and removes the lowest-scoring ones until exactly N remain.
4. **Recover.** The survivors are optimized for the remaining 30,000 updates.

The primary comparison applies four consecutive cycles. Both models then undergo 100,000 updates with growth and pruning disabled, testing whether the refresh advantage persists after population changes stop.

The remaining sections define the removal-score, pruning-schedule, and parent-selection variants. Sections 4.3 and 4.4 define which primitives are removed and in what order. Section 4.5 defines how growth parents are selected. The primary comparison uses a $3N$ peak and four cycles. We vary the peak across $\{N, 2N, 3N, 5N\}$

in a single cycle to select it, and we extend the four-cycle schedule to fifteen cycles to locate the point at which repeated refresh stops improving. Section 5.3 reports the measured effects of both choices.

4.3 Signed Removal Score

The GaussianPOP criterion of Section 2.4.1 measures a magnitude. Equation 2.17 gives the squared change that removing primitive k would produce in the render, and that quantity is computed from the render alone. It cannot distinguish a removal that moves the render toward the captured photograph from one that moves it away, because the photograph is never an input.

We extend the criterion to use that photograph. Let $\mathbf{d}_k = T_k \alpha_k (\mathbf{c}_k - \mathbf{b}_{k+1})$ be the contribution of primitive k at a pixel; Equation 2.17 squares its magnitude. Let $\mathbf{r} = C_{\text{render}} - C_{\text{gt}}$ be the current residual at that pixel against the captured image. Removing k replaces the residual $\mathbf{r}$ with $\mathbf{r} - \mathbf{d}_k$. Expanding the resulting change in squared error and aggregating over views and pixels gives the signed score

$$\Delta L_k = \sum_{v,p} \|\mathbf{d}_k\|^2 - 2\lambda_r \sum_{v,p} (\mathbf{r} \cdot \mathbf{d}_k), \quad (4.1)$$

where λ_r weights the residual term. The first sum is exactly the GaussianPOP importance of Equation 2.18. The second sum is the contribution of this thesis, and its sign carries the new information. When $\mathbf{r}$ and $\mathbf{d}_k$ point in the same direction, primitive k is contributing to an error that already exists, and removing it cancels part of that error. When they oppose, primitive k is supplying signal the render is missing, and removing it makes the residual worse even if its magnitude is large.

Setting $\lambda_r = 0$ recovers GaussianPOP exactly, and $\lambda_r = 1$ gives the exact single-removal change in squared color error when all remaining primitive parameters

are fixed. We evaluate $\lambda_r \in \{0, 0.25, 0.5, 1\}$ and adopt $\lambda_r = 0.25$ for the five-scene comparison in Section 5.4. Note that λ_r is distinct from the loss weight λ of Equation 2.13.

The residual term requires the captured image, so unlike GaussianPOP, the signed score is not ground-truth-free. We evaluate whether this additional information improves final pruning quality.

4.4 Pruning Schedule

Given a score, four further decisions determine how a population is reduced to N . Each comparison changes one decision while holding the others fixed.

Rescore frequency. Pruning uses C rounds of scoring, deletion, and recomputation. We evaluate six values of C : 1, 2, 4, 8, 16, and 32. At $C = 1$, every primitive is ranked once and the entire deletion target is removed in a single pass. At $C = 32$, each round removes roughly 3% of the target and then rescores against the model that now exists. No optimization occurs between rounds. Each value of C then receives 10,000 recovery updates, and we report quality both immediately after pruning and after that recovery.

Staged prune and recover. The end-only schedule completes 32 pruning rounds before 40,000 recovery updates. The staged schedule divides them into four groups of eight rounds, each followed by 10,000 recovery updates. The schedules contain the same 32 pruning rounds and 40,000 recovery updates but order them differently. Each recovery block updates the survivors before the next group of pruning rounds is scored. Figure 4.2 compares the operation order.

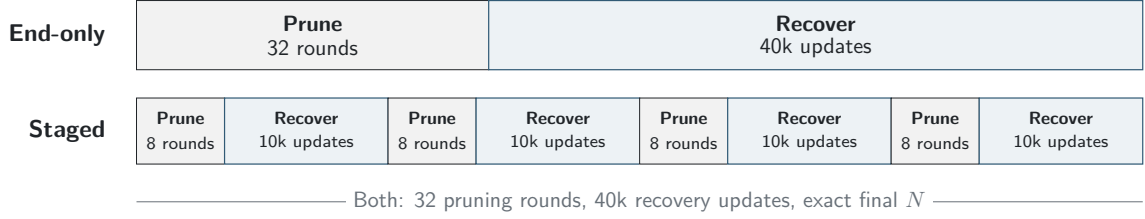


Figure 4.2: Both schedules use 32 pruning rounds, 40,000 recovery updates, and the same exact final count. Staging places 10,000 recovery updates after each group of eight pruning rounds.

Visibility threshold. With threshold K , primitives visible in fewer than K training views are prioritized for deletion. The motivation is that a primitive visible to only a few cameras may represent transient rather than persistent scene content. Here, transient content denotes moving objects or temporary occluders captured in only a subset of images, whereas persistent content denotes static scene structure supported across multiple viewpoints. We evaluate $K \in \{5, 10, 20\}$ against standard pruning with this rule disabled.

View aggregation. Equation 2.18 sums the per-view removal error over all training views with equal weight. The alternative retains only the largest half of the per-view scores before summing, excluding the lower-scoring view contributions.

Rescoring updates importance after deletion without optimizing the survivors. Staging additionally optimizes the survivors before later scores are computed. Section 5.4 reports their measured effects.

4.5 Parent Selection During Refresh

Deterministic selection favors the strongest eligible positional-gradient scores at each growth event. Across repeated refresh cycles, this can direct growth toward similar high-gradient regions. We test whether sampling from the same eligible set instead improves the final model by varying the parents selected for growth.

Both schedules retain the positional-gradient criterion of Equation 2.14 and standard cloning and splitting. Deterministic selection chooses the highest-scoring eligible parents. Probabilistic selection samples eligible parents without replacement with probability proportional to the square root of their positional-gradient score. This weighting reduces the concentration on the largest scores while retaining the same eligibility criterion.

Both schedules apply five $N \rightarrow 3N \rightarrow N$ cycles. Each cycle uses 5,000 growth updates, 5,000 stabilization updates, 32 pruning rounds, and 30,000 recovery updates. The schedules differ only in parent selection.

Chapter 5 evaluates the signed score, pruning schedules, parent selection, and population refresh using the matched checkpoints and budgets described in Section 4.1.

Chapter 5

Results

This chapter evaluates whether population refresh improves a trained model at a fixed final count and how peak size, cycle count, pruning, and parent selection affect held-out reconstruction quality.

5.1 Population Refresh Against a Fixed Population

The primary experiment evaluates whether replacing primitives during continued optimization improves a trained model at a fixed final population size. It compares four refresh cycles at a peak of $3N$ with optimization of the original population. Section 5.3 examines how peak size and cycle count affect the result.

Table 5.1 reports both schedules at 160,000 updates and after refinement at 260,000 updates. The fixed baseline optimizes the original population throughout. The refreshed model completes four cycles at 160,000 updates; both models then receive 100,000 refinement updates with growth and pruning disabled. After refinement, refresh improves the mean by .3028 dB PSNR and .00382 SSIM, reduces LPIPS by .00286, and improves 14 of 15 scene-metric pairs; LPIPS on Truck is the exception.

Table 5.1: PSNR before and after the 100,000-update refinement stage for both matched schedules. Deltas compare all metrics on the refreshed and fixed models after refinement.

Scene	Fixed baseline (PSNR)		Refreshed model (PSNR)		Final change		
	Before (160k)	After (260k)	Before (160k)	After (260k)	Δ PSNR	Δ SSIM	Δ LPIPS
Barn	28.1323	28.1854	28.3839	28.4882	+.3027	+.00262	-.00159
Caterpillar	21.3426	21.4832	21.5088	21.5289	+.0456	+.00329	-.00214
Ignatius	22.4007	22.7616	22.9148	22.9474	+.1859	+.00287	-.00068
Meetingroom	26.3937	26.6553	27.0674	27.2951	+.6398	+.00962	-.01423
Truck	25.5694	25.5616	25.7677	25.9018	+.3402	+.00073	+.00436
Mean	24.7677	24.9294	25.1285	25.2323	+. 3028	+.00382	-.00286

Figure 5.1 reports absolute held-out metrics at every matched checkpoint. The refreshed model begins to outperform the fixed baseline during the refresh cycles and retains higher PSNR and SSIM and lower LPIPS through refinement.

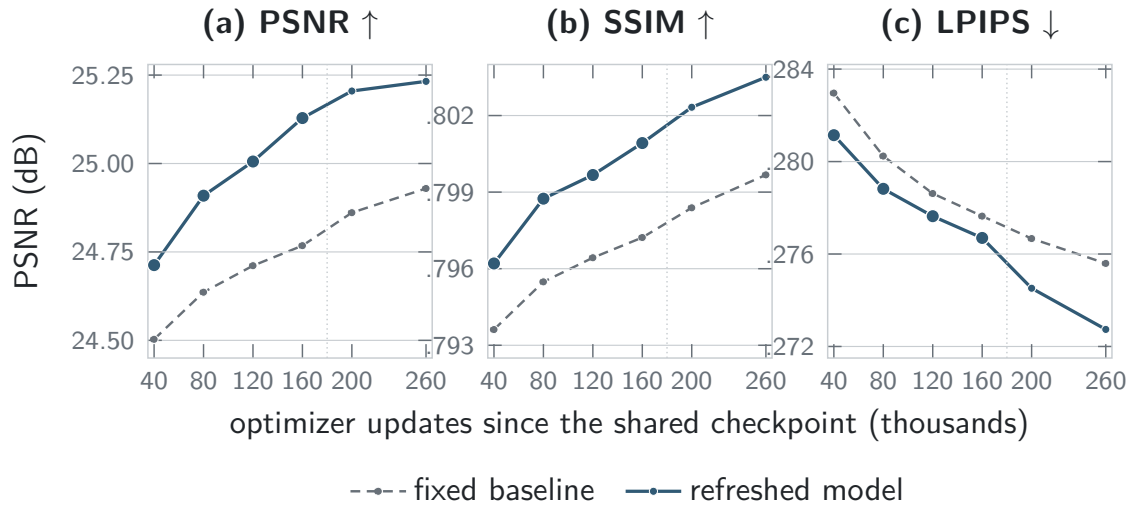


Figure 5.1: Absolute five-scene means at matched checkpoints. Large dots mark refresh-cycle endpoints; small dots mark optimization-only checkpoints. The refreshed model retains higher PSNR and SSIM and lower LPIPS through refinement, ending .3028 dB above the fixed baseline in PSNR.

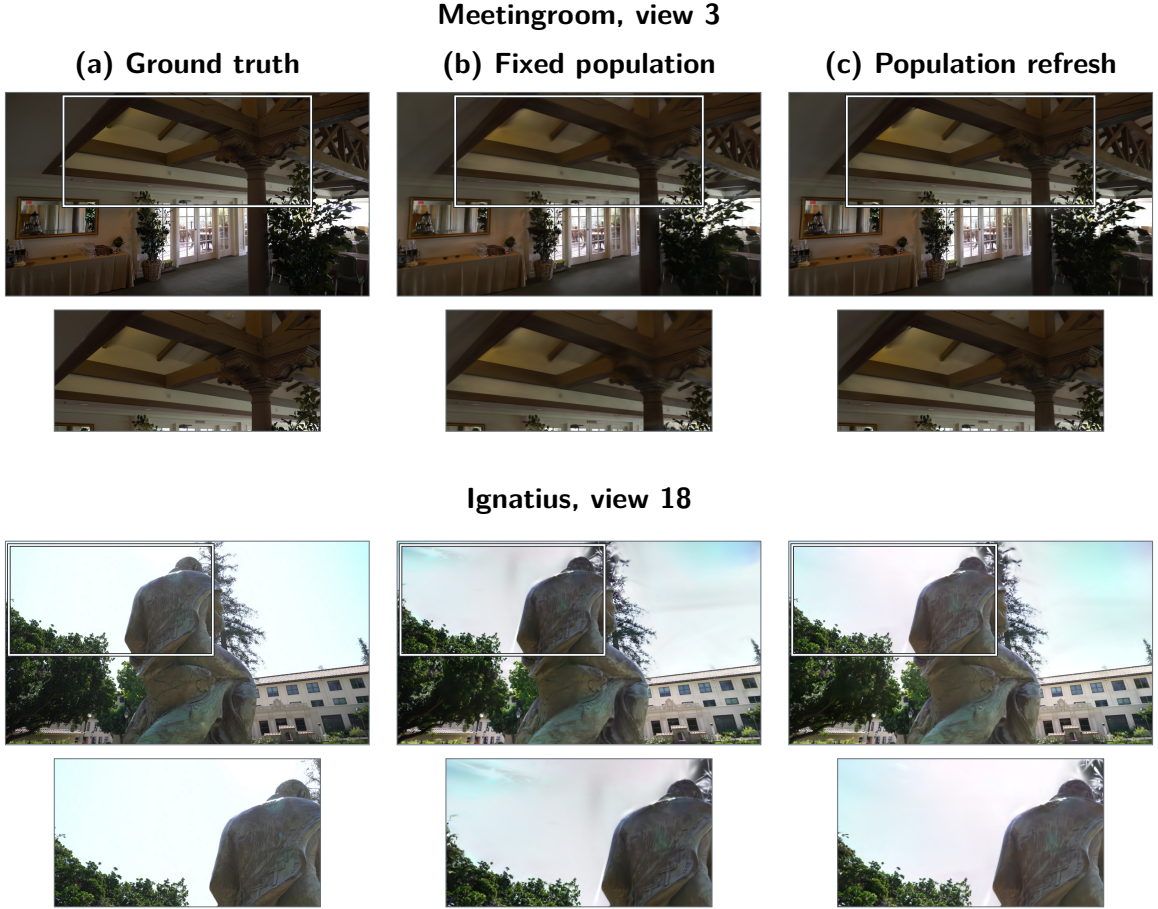


Figure 5.2: Qualitative comparisons at the matched final count after refinement. Population refresh better preserves ceiling beams and column detail in Meetingroom and reduces the broad sky artifact in Ignatius. Boxes mark enlarged regions.

5.2 Spatial Occupancy Analysis

To measure spatial redistribution at a fixed final count, we project the Gaussian centers of the refreshed and fixed-population models into each held-out camera, bin them into an 18×32 grid, and compare occupancy cell by cell. The 212 held-out views produce 122,112 cells.

Table 5.2 groups cells by occupancy change. Cells that gain six or more primi-

tives have a mean change of +0.48 dB, and 62% improve. Cells that lose five or more primitives have a mean change of +0.17 dB, and 52% improve. Mean PSNR change is positive in every occupancy group.

Table 5.2: Mean per-cell PSNR change is positive in every occupancy group. Among cells gaining six or more primitives, 62% improve. Cells are 18×32 image bins over 212 held-out views.

Cells that	Count	Δ PSNR $\uparrow$	Share improved
lost 5 or more	50,662	+0.17	52%
lost 1 to 4	18,800	+0.41	53%
did not change	6,861	+0.48	53%
gained 1 to 5	17,669	+0.37	56%
gained 6 or more	28,120	+0.48	62%

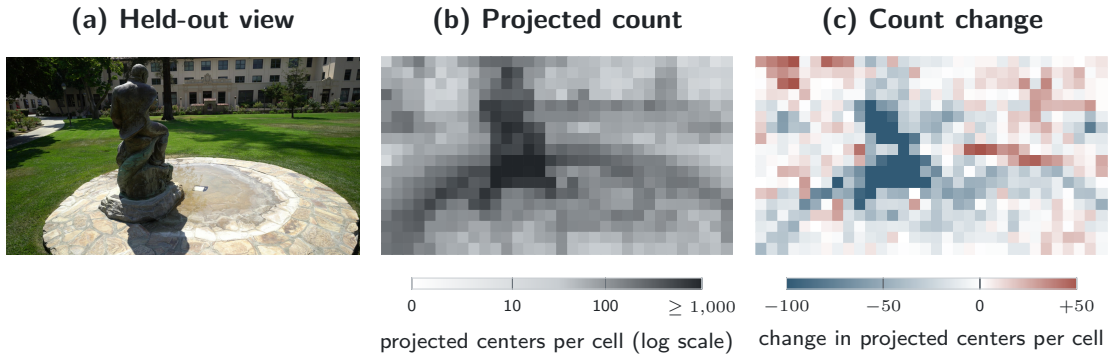


Figure 5.3: Ignatius held-out view 16. Panel (b) shows fixed-population projected-center counts; panel (c) shows refresh minus fixed population, with color clipped at -100 and $+50$. Table 5.2 aggregates this 18×32 -cell comparison over all 212 held-out views.

5.3 Peak Population and Cycle Count

5.3.1 Peak Size

Table 5.3 compares the fixed baseline with single refreshes that reach peaks of $2N$, $3N$, and $5N$ before returning to N . All three refresh configurations increase mean PSNR relative to the fixed population. A peak of $3N$ gives the highest PSNR and SSIM, while $2N$ gives the lowest LPIPS.

Table 5.3: Held-out metrics after one 40,000-update schedule. Refresh expands to the listed peak and returns to N ; the fixed baseline remains at N .

Peak	Procedure	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
N	fixed population, no refresh	24.5029	.79362	.28296
$2N$	one $N \rightarrow 2N \rightarrow N$ refresh	24.6496	.79611	.28073
$3N$	one $N \rightarrow 3N \rightarrow N$ refresh	24.7127	.79620	.28114
$5N$	one $N \rightarrow 5N \rightarrow N$ refresh	24.6192	.79374	.28345

A peak of $5N$ increases PSNR by .1163 dB over the fixed population, but its SSIM is nearly unchanged and its LPIPS is higher. Subsequent experiments use $3N$, which gives the highest mean PSNR and SSIM. The single $3N$ refresh changes the means by +.2098 dB PSNR, +.00258 SSIM, and $-.00182$ LPIPS.

5.3.2 Cycle Count

Figure 5.4 shows the five-scene means over fifteen consecutive refresh cycles. The cycle-ten decline is concentrated in Caterpillar and Meetingroom. At cycle eleven, SSIM and LPIPS exceed their cycle-nine values, while PSNR remains lower.

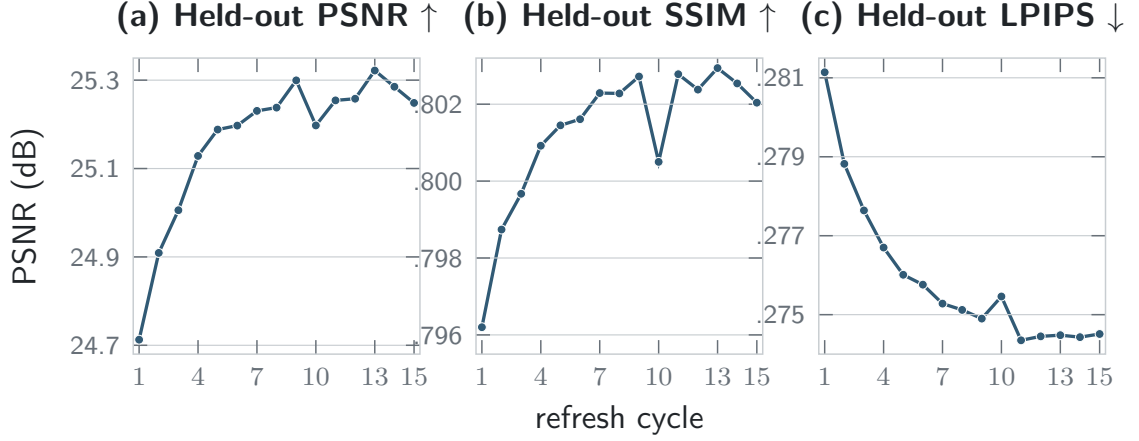


Figure 5.4: Five-scene held-out means over fifteen cycles. The three metrics improve overall through cycle nine. From cycles ten through fifteen, PSNR and SSIM fluctuate, while LPIPS remains near its cycle-eleven minimum.

5.4 Pruning Components

5.4.1 Signed Removal Score

At $\lambda_r = 1$, the signed score of Section 4.3 equals the single-removal change when all remaining parameters are fixed; $\lambda_r = 0$ reproduces GaussianPOP. Table 5.4 evaluates its use in bulk pruning.

Every nonzero residual weight worsens SSIM and LPIPS on all five scenes. Mean PSNR is nearly unchanged at $\lambda_r = 0.25$ and declines at larger weights.

Table 5.4: Recovered mean changes relative to GaussianPOP ($\lambda_r = 0$) at matched final count.

Weight	Δ PSNR	Δ SSIM	Δ LPIPS	Scenes improved (P/S/L)
$\lambda_r = 0.25$	+0.0025	-.00029	+0.00046	2/5, 0/5, 0/5
$\lambda_r = 0.50$	-.0121	-.00064	+0.00085	2/5, 0/5, 0/5
$\lambda_r = 1.00$	-.0880	-.00289	+0.00380	0/5, 0/5, 0/5

The signed score is exact only for one removal with the remaining parameters fixed. Because bulk deletion changes transmittance and surviving contributions, scores computed before deletion do not represent the effect of later removals. This mismatch may contribute to the decline as λ_r increases.

5.4.2 Rescore Frequency

Table 5.5 varies the number of score, delete, and recompute rounds. Immediately after pruning, PSNR increases from 24.0072 at $C = 1$ to 24.0330 at $C = 32$, while SSIM increases and LPIPS decreases. By $C = 4$, the immediate metrics are within .0048 dB PSNR, .00008 SSIM, and .00003 LPIPS of $C = 32$.

Table 5.5: Immediately after pruning, metrics improve as C increases, but after 10,000 recovery updates all six settings are within .013 dB PSNR and show no consistent ordering.

C	Immediately after pruning			After 10k recovery		
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
1	24.0072	.78260	.29258	24.3962	.79083	.28599
2	24.0221	.78293	.29227	24.3947	.79077	.28605
4	24.0282	.78309	.29214	24.4064	.79093	.28597
8	24.0316	.78313	.29213	24.3957	.79082	.28603
16	24.0327	.78316	.29212	24.4009	.79089	.28603
32	24.0330	.78317	.29211	24.3936	.79089	.28605

After recovery, all six settings are within .013 dB PSNR and no longer follow the immediate ordering; $C = 32$ is .0026 dB below $C = 1$.

5.4.3 Staged Pruning and Recovery

This comparison evaluates recovery placed between pruning stages against recovery performed only after pruning. Dividing pruning into four stages, with recovery between them, increases mean PSNR by .0466 dB.

Table 5.6: The four-stage schedule changes the mean metrics by $+0.0466$ dB PSNR, $+0.00053$ SSIM, and -0.00085 LPIPS at identical total recovery and final count. Most of the PSNR difference comes from Meetingroom.

Scene	One stage PSNR $\uparrow$	Four stages PSNR $\uparrow$	Δ PSNR	Δ SSIM	Δ LPIPS
Barn	28.0282	28.0196	-0.0085	$+0.00091$	-0.00040
Caterpillar	21.2574	21.2623	$+0.0049$	-0.00070	-0.00133
Ignatius	22.2096	22.2149	$+0.0053$	$+0.00058$	-0.00018
Meetingroom	26.1122	26.3666	$+0.2544$	$+0.00217$	-0.00239
Truck	25.4388	25.4157	-0.0231	-0.00032	$+0.00007$
Mean	24.6092	24.6558	$+0.0466$	$+0.00053$	-0.00085

Meetingroom drives the mean PSNR increase, while Barn and Truck decrease. The benefit of staged recovery therefore varies across scenes.

5.4.4 Visibility Priority and View Aggregation

Table 5.7 shows results after prioritizing for deletion primitives visible in fewer than K training views. At $K = 5$ and $K = 10$, absolute changes relative to standard pruning do not exceed $.0041$ dB PSNR, $.00006$ SSIM, or $.00006$ LPIPS. At $K = 20$, PSNR decreases by $.0586$ dB, SSIM decreases by $.00078$, and LPIPS increases by $.00030$.

Table 5.7: Prioritizing low-visibility primitives does not improve all three mean metrics at $K = 5$ or $K = 10$ and worsens all three at $K = 20$.

Priority rule	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
Standard pruning	24.3336	.79019	.28662
$K = 5$	24.3295	.79013	.28665
$K = 10$	24.3334	.79024	.28656
$K = 20$	24.2750	.78941	.28693

None of the tested visibility thresholds improves all three mean metrics. Replacing the all-view sum of Equation 2.18 with the largest half of the per-view scores changes the means by $-.0048$ dB PSNR, $+.00003$ SSIM, and $+.00004$ LPIPS after recovery.

5.5 Parent Selection During Refresh

Table 5.8 compares deterministic and probabilistic parent selection after five cycles while retaining the positional-gradient criterion and standard growth operations.

Table 5.8: Held-out metrics after five matched $N \rightarrow 3N \rightarrow N$ cycles. The schedules differ only in parent selection.

Scene	PSNR $\uparrow$		Δ PSNR	Δ SSIM	Δ LPIPS
	Deterministic	Probabilistic			
Barn	28.3723	28.4078	+0.0355	+0.00012	−0.00065
Caterpillar	21.5201	21.5043	−0.0158	−0.00053	+0.00024
Ignatius	22.8846	22.9876	+0.1029	+0.00102	−0.00026
Meetingroom	27.3213	27.2428	−0.0784	−0.00144	+0.00028
Truck	25.8427	25.8353	−0.0074	+0.00011	−0.00020
Mean	25.1882	25.1956	+0.0074	−0.00015	−0.00012

Probabilistic selection changes the mean metrics by +0.0074 dB PSNR, −0.00015 SSIM, and −0.00012 LPIPS. Mean PSNR and LPIPS improve, whereas mean SSIM decreases; scene-level changes also vary in sign.

5.6 Summary of Findings

Table 5.9 summarizes the main controlled comparisons.

Table 5.9: Population refresh improves all three mean metrics. Most pruning and parent-selection comparisons do not.

Intervention	Compared against	Δ PSNR	Δ SSIM	Δ LPIPS	Outcome
Four cycles at 260k	fixed population	+.3028	+.00382	−.00286	persistent increase
Four cycles at 160k	fixed population	+.3608	+.00370	−.00094	largest PSNR increase
One $3N$ refresh	fixed population	+.2098	+.00258	−.00182	all means improve
Staged prune and recover	one stage	+.0466	+.00053	−.00085	small mean increase
Signed score $\lambda_r = .25$	GaussianPOP	+.0025	−.00029	+.00046	mixed
$C = 32$, recovered	$C = 1$	−.0026	+.00006	+.00006	mixed
Top- k 50% aggregation	all-view sum	−.0048	+.00003	+.00004	mixed
Visibility $K = 10$	standard pruning	−.0002	+.00005	−.00006	mixed
Probabilistic selection	deterministic selection	+.0074	−.00015	−.00012	mixed
Visibility $K = 20$	standard pruning	−.0586	−.00078	+.00030	all means worsen

Most pruning comparisons do not improve all three recovered metrics. The rescore-frequency ordering observed immediately after pruning is absent after recovery. Population refresh improves all three means and produces larger changes than the tested pruning and parent-selection variants.

Chapter 6

Discussion and Conclusion

6.1 Discussion

Continued optimization alone does not reproduce the refresh gain. Each tested refresh peak improves mean PSNR, but $3N$ gives the highest PSNR and SSIM. The peak-size and cycle-count experiments show that increasing either quantity does not consistently improve reconstruction quality.

Cells gaining at least six projected centers improve most consistently: 62% have higher PSNR, the largest share of any group. Mean PSNR also increases in every occupancy group, so the benefit is not limited to cells receiving additional primitives.

The pruning comparisons yield two findings: the rescore-frequency ordering observed immediately after pruning is absent after recovery, and the signed residual term does not improve the three metrics together. Population refresh, which introduces new candidates, produces larger improvements than the tested changes to removal ranking.

6.2 Future Work

The five-scene Tanks and Temples evaluation [8] should be extended across datasets and capture conditions. Camera coverage, scene extent, and initialization density may change how redundancy is distributed within a fixed Gaussian budget.

Future work should replace a fixed cycle count with a dynamic stopping mechanism. Candidate signals include the fraction of newly spawned primitives retained after pruning and the improvement in the training objective per cycle.

Population refresh should also be integrated into initial training. Comparing it with standard adaptive density control over a common training horizon would test whether it complements densification without requiring a separate post-training phase.

A direct mechanism analysis should assign persistent identifiers, record parent-child links, and store positions at each cycle. These additions would track which primitives survive, where replacements originate, and how far they move.

6.3 Conclusion

This thesis examined whether population replacement after training improves a 3DGS model beyond continued optimization. It also evaluated removal scoring, pruning schedule, peak population, cycle count, and parent selection.

The main result is the controlled population-refresh comparison. After four $N \rightarrow 3N \rightarrow N$ cycles followed by 100,000 refinement updates, refresh increases the five-scene mean by .3028 dB PSNR and .00382 SSIM, reduces LPIPS by .00286, and improves 14 of 15 scene-metric pairs. Repeated refresh improves metrics through cycle nine; later cycles fluctuate without sustained gains.

Among the pruning variants, only staged recovery improves all three mean metrics, and the parent-selection comparison is mixed. Within the five evaluated scenes, population refresh produces the largest and most consistent gains. Controlled replacement therefore improves reconstruction without increasing the final Gaussian count.

Bibliography

- [1] M. Tancik, V. Casser, X. Yan, S. Pradhan, B. Mildenhall, P. P. Srinivasan, J. T. Barron, and H. Kretzschmar, “Block-NeRF: Scalable large scene neural view synthesis,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 8248–8258, 2022.
- [2] S. Qian, T. Kirschstein, L. Schoneveld, D. Davoli, S. Giebenhain, and M. Nießner, “GaussianAvatars: Photorealistic head avatars with rigged 3D gaussians,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 20299–20309, 2024.
- [3] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, “NeRF: Representing scenes as neural radiance fields for view synthesis,” in *European Conference on Computer Vision (ECCV)*, 2020.
- [4] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, “3D Gaussian splatting for real-time radiance field rendering,” *ACM Transactions on Graphics*, vol. 42, no. 4, 2023.
- [5] Z. Fan, K. Wang, K. Wen, Z. Zhu, D. Xu, and Z. Wang, “LightGaussian: Unbounded 3D gaussian compression with 15x reduction and 200+ FPS,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [6] S. S. Mallick, R. Goel, B. Kerbl, M. Steinberger, F. V. Carrasco, and F. De La Torre, “Taming 3DGS: High-quality radiance fields with limited resources,” in *SIGGRAPH Asia Conference Papers*, 2024.
- [7] S. Lee, Y.-G. Kim, S. Sasse, T. M. Borges, Y. Sánchez, E.-S. Ryu, T. Schierl, and C. Hellge, “GaussianPOP: Principled simplification framework for compact 3D gaussian splatting via error quantification,” *arXiv preprint arXiv:2602.06830*, 2026.
- [8] A. Knapitsch, J. Park, Q.-Y. Zhou, and V. Koltun, “Tanks and temples: Benchmarking large-scale scene reconstruction,” *ACM Transactions on Graphics*, vol. 36, no. 4, 2017.
- [9] J. L. Schönberger and J.-M. Frahm, “Structure-from-motion revisited,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [10] N. Max, “Optical models for direct volume rendering,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 1, no. 2, pp. 99–108, 1995.
- [11] M. Zwicker, H. Pfister, J. van Baar, and M. Gross, “EWA splatting,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 8, no. 3, pp. 223–238, 2002.
- [12] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *International Conference on Learning Representations (ICLR)*, 2015.
- [13] G. Fang and B. Wang, “Mini-splatting: Representing scenes with a constrained number of gaussians,” in *European Conference on Computer Vision (ECCV)*, 2024.

- [14] M. Niemeyer, F. Manhardt, M.-J. Rakotosaona, M. Oechsle, D. Duckworth, R. Gosula, K. Tateno, J. Bates, D. Kaeser, and F. Tombari, “RadSplat: Radiance field-informed gaussian splatting for robust real-time rendering with 900+ FPS,” in *International Conference on 3D Vision (3DV)*, 2025.
- [15] S. Niedermayr, J. Stumpfegger, and R. Westermann, “Compressed 3D gaussian splatting for accelerated novel view synthesis,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [16] A. Hanson, A. Tu, V. Singla, M. Jayawardhana, M. Zwicker, and T. Goldstein, “PUP 3D-GS: Principled uncertainty pruning for 3D gaussian splatting,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [17] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: From error visibility to structural similarity,” *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.
- [18] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, “The unreasonable effectiveness of deep features as a perceptual metric,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [19] W. Morgenstern, F. Barthel, A. Hilsmann, and P. Eisert, “Compact 3d scene representation via self-organizing gaussian grids,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [20] K. L. Navaneet, K. Pourahmadi Meibodi, S. Abbasi Koohpayegani, and H. Pirsiavash, “CompGS: Smaller and faster gaussian splatting with vector quantization,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [21] S. Xie, W. Zhang, C. Tang, Y. Bai, R. Lu, S. Ge, and Z. Wang, “MesonGS: Post-training compression of 3D gaussians via efficient attribute transformation,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [22] S. Girish, K. Gupta, and A. Shrivastava, “EAGLES: Efficient accelerated 3D gaussians with lightweight encodings,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [23] Y. Zhang, W. Jia, W. Niu, and M. Yin, “GaussianSpa: An optimizing-sparsifying simplification framework for compact and high-quality 3D gaussian splatting,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [24] Y. Liu, Z. Zhong, Y. Zhan, S. Xu, and X. Sun, “MaskGaussian: Adaptive 3D gaussian representation from probabilistic masks,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [25] S. Rota Bulò, L. Porzi, and P. Kotschieder, “Revising densification in gaussian splatting,” in *European Conference on Computer Vision (ECCV)*, 2024. arXiv:2404.06109.
- [26] Z. Zeng, Y. Wang, L. Ju, and T. Guan, “Frequency-aware density control via reparameterization for high-quality rendering of 3D gaussian splatting,” in *AAAI Conference on Artificial Intelligence*, 2025. arXiv:2503.07000.
- [27] J. Zhang, F. Zhan, M. Xu, S. Lu, and E. Xing, “FreGS: 3D gaussian splatting with progressive frequency regularization,” in *IEEE Conference on Computer*

- Vision and Pattern Recognition (CVPR)*, 2024. arXiv:2403.06908.
- [28] B. Huang, Z. Liu, and N. Wong, “Decomposing densification in gaussian splatting for faster 3D scene reconstruction,” *arXiv preprint arXiv:2507.20239*, 2025.
 - [29] P. Wang, Y. Wang, D. Wang, S. Mohan, Z. Fan, L. Wu, R. Cai, Y.-Y. Yeh, Z. Wang, Q. Liu, and R. Ranjan, “Steepest descent density control for compact 3D gaussian splatting,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
 - [30] Z. Zhou, Y.-J. Xiong, J.-C. Zhang, C.-M. Xia, X. Qiu, and H. Zhan, “Gradient-direction-aware density control for 3D gaussian splatting,” in *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2508.10507.
 - [31] H. Yang, C. Zhang, W. Wang, M. Volino, A. Hilton, L. Zhang, and X. Zhu, “Improving gaussian splatting with localized points management,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. arXiv:2406.04251.
 - [32] D. Kotovenko, O. Grebenkova, and B. Ommer, “EDGS: Eliminating densification for efficient convergence of 3DGS,” *arXiv preprint arXiv:2504.13204*, 2025.
 - [33] J. C. Lee, J. H. Ko, and E. Park, “Optimized minimal 3D gaussian splatting,” *arXiv preprint arXiv:2503.16924*, 2025.
 - [34] B. Zhang, C. Fang, R. Shrestha, Y. Liang, X.-X. Long, and P. Tan, “RaDeGS: Rasterizing depth in gaussian splatting,” *ACM Transactions on Graphics*, vol. 45, no. 2, 2026. arXiv:2406.01467, 2024.

Vita**Matthew Mammano****Education**

Stevens Institute of Technology, Hoboken, NJ
M.S. in Machine Learning, August 2026
B.S. in Computer Science, May 2025

**Professional
Experience**

PTC Therapeutics, Inc., Warren, NJ
Senior Machine Learning Engineer, May 2026–present
AI & Machine Learning Fellow, May 2025–May 2026

**Research
Interests**

Machine learning, computer vision, 3D reconstruction,
and efficient 3D scene representations.